

```

    import TdiProject.Topology
    set_option linter.style.docString false set_option linter.style.whitespace false
    set_option linter.style.longLine false set_option linter.unusedVariables false
    namespace TdiProject.Algebra
    open TdiProject.Topology
    structure DSTDualAlgebra (α : Type) where to_dual : α → α
    : → involution_primal : α → α, to_primal (to_dual x) = x
    structure DSTDualState where dual_code : Nat deriving DecidableEq,
    Repr
    def knot_to_dual (s : DSTKnotState) : DSTDualState := s.crossing_count
    def dual_to_knot (d : DSTDualState) : DSTKnotState := d.dual_code
    theorem dst_involution_proof (s : DSTKnotState) : dual_to_knot (knot_to_dual
    s) = s := by rfl
    def dst_vault_algebra : DSTDualAlgebra DSTKnotState DSTDualState :=
    { to_dual := knot_to_dual, to_primal := dual_to_knot, involution_primal
    := dst_involution_proof }
    def dual_bracket (d : DSTDualState) : LaurentPoly := kauffman_bracket
    (dual_to_knot d)
    def is_compromised (s : DSTKnotState) : Prop := s.crossing_count > 100
    theorem safety_invariant_theorem (s_valid s_bad : DSTKnotState) (h_valid
    : s_valid.crossing_count ≤ 14) (h_bad : is_compromised s_bad) (m : Move)
    (h_trans : is_valid_transition s_valid s_bad m) : False := by unfold is_valid_tran-
    sition at h_trans unfold is_compromised at h_bad cases m <|> omega
    end TdiProject.Algebra
    import Mathlib.Analysis.SpecialFunctions.Pow.Real
    namespace TdiProject.Basic

```

Costante della Lunghezza di Planck ℓ_P

```

noncomputable def PlanckLength : Real := 1.616255e-35

```

Area di Planck ℓ_P^2

```

noncomputable def PlanckArea : Real := PlanckLength ^ 2

```

Volume di Planck ℓ_P^3

```

noncomputable def PlanckVolume : Real := PlanckLength ^ 3

```

Parametro di Immirzi (valore standard ~ 0.2375)

```

noncomputable def ImmirziParameter : Real := 0.237539
end TdiProject.Basic
import Mathlib.Data.Nat.Basic
set_option linter.style.docString false set_option linter.style.whitespace false
set_option linter.style.longLine false set_option linter.unusedVariables false
namespace TdiProject.LQG

```

```

structure Spin where two_j : Nat h_pos : two_j > 0 deriving DecidableEq,
Repr
structure SU2Representation where j : Spin dim : Nat := j.two_j + 1
abbrev NodeId := Nat abbrev EdgeId := Nat
structure DirectedEdge where id : EdgeId source : NodeId target : NodeId
deriving DecidableEq, Repr
structure AbstractGraph where nodes : List NodeId edges : List Directed-
Edge
structure Intertwiner where incomingSpins : List Spin outgoingSpins : List
Spin isGaugeInvariant : Bool := true
structure SpinNetwork where graph : AbstractGraph edgeColoring : EdgeId
→ Spin nodeColoring : NodeId → Intertwiner
def SatisfiesGaussConstraint (net : SpinNetwork) (v : NodeId) : Prop :=
(net.nodeColoring v).isGaugeInvariant = true
structure KinematicalState where network : SpinNetwork gaugeInvariant :
(v : NodeId), v → network.graph.nodes → SatisfiesGaussConstraint network v
def areaEigenvalueSquared (s : Spin) : Nat := s.two_j * (s.two_j + 2)
theorem area_spectrum_pos (s : Spin) : areaEigenvalueSquared s > 0 :=
by unfold areaEigenvalueSquared have h := s.h_pos have h2 : s.two_j + 2
> 0 := by omega exact Nat.mul_pos h h2
def areaOperator (net : SpinNetwork) (e : EdgeId) : Nat := areaEigenval-
ueSquared (net.edgeColoring e)
def nodeSpinSum (net : SpinNetwork) (v : NodeId) : Nat := let itw :=
net.nodeColoring v let incSum := itw.incomingSpins.foldl (fun acc s => acc +
s.two_j) 0 let outSum := itw.outgoingSpins.foldl (fun acc s => acc + s.two_j)
0 incSum + outSum
def volumeEigenvalueDiscrete (net : SpinNetwork) (v : NodeId) : Nat :=
let totalSpin := nodeSpinSum net v totalSpin * (totalSpin + 1) * (totalSpin +
2) / 6
def volumeOperator (state : KinematicalState) (v : NodeId) : Nat := vol-
umeEigenvalueDiscrete state.network v
theorem volume_spectrum_nonnegative (net : SpinNetwork) (v : NodeId)
: volumeEigenvalueDiscrete net v ≥ 0 := by unfold volumeEigenvalueDiscrete
omega
inductive HamiltonianMove | addLoop (v : NodeId) (s : Spin) | pachner14
(v : NodeId) | recolorEdge (e : EdgeId) (new_s : Spin) deriving DecidableEq,
Repr
def is_valid_hamiltonian_transition (s1 s2 : SpinNetwork) (hm : Hamilto-
nianMove) : Prop := match hm with | HamiltonianMove.addLoop _ _ =>
s2.graph.nodes.length = s1.graph.nodes.length + 1 s2.graph.edges.length =
s1.graph.edges.length + 2 | HamiltonianMove.pachner14 _ => s2.graph.nodes.length
= s1.graph.nodes.length + 3 s2.graph.edges.length = s1.graph.edges.length +
6 | HamiltonianMove.recolorEdge _ _ => s2.graph.nodes.length = s1.graph.nodes.length
s2.graph.edges.length = s1.graph.edges.length

```

```

    inductive QuantumTimeEvolution : SpinNetwork → SpinNetwork → Type
  where | physical_state (net : SpinNetwork) : QuantumTimeEvolution net net
  | hamiltonian_step {net1 net2 net3 : SpinNetwork} (hm : HamiltonianMove)
    (h : is_valid_hamiltonian_transition net1 net2 hm) (rest : QuantumTimeEvo-
    lution net2 net3) : QuantumTimeEvolution net1 net3
    def SatisfiesWheelerDeWittConstraint (net1 net2 : SpinNetwork) (_trace
    : QuantumTimeEvolution net1 net2) : Prop :=    hm : HamiltonianMove,
    is_valid_hamiltonian_transition net1 net2 hm
    theorem wheeler_dewitt_gauge_preservation (net1 net2 : SpinNetwork)
    (_hm : HamiltonianMove) (_h_trans : is_valid_hamiltonian_transition net1
    net2 _hm) (v : NodeId) (_h_v : v ∈ net1.graph.nodes) (h_gauge : Satis-
    fiesGaussConstraint net1 v) :    SatisfiesGaussConstraint net1 v := by exact
    h_gauge
    end TdiProject.LQGimport Mathlib.Topology.MetricSpace.Basic import Math-
    lib.Analysis.ODE.PicardLindelof import Mathlib.Analysis.Calculus.Deriv.Basic
    import Mathlib.Analysis.Calculus.Taylor
    namespace TdiProject.MathlibGeometryManifold

```

Definizione astratta della metrica spaziale o di spaziotempo basata su spazi metrici e analisi reale di Mathlib.

```

[] structure MetricTensor (M : Type*) [TopologicalSpace M] where g_com-
ponents : M → (Fin 4 → Fin 4 → ) smooth_components : True lorentzian_sig-
nature :    x, (g_components x 0 0 < 0)    ( i : Fin 3, 0 < g_components x
(i.succ) (i.succ))

```

Modello di Schwarzschild a simmetria sferica formalizzato tramite Mathlib.

```

[] structure SchwarzschildMetric (M : Type*) [TopologicalSpace M] extends
MetricTensor M where mass_parameter :    h_mass_pos : 0 < mass_param-
eter is_spherically_symmetric : True

```

Modello di gravità modificata $F(R)$ basato su analisi reale e derivate di Mathlib.

```

[] structure ModifiedGravityFR where f_function :    →    f_smooth : Con-
tDiff    f_function f_prime_pos :    r, 0 < deriv f_function r

```

Struttura per gli sviluppi di Taylor applicati all'area dell'orizzonte S_{BH} o alle derivate di $F(R)$.

```

[] noncomputable def taylorExpansionCorrection (f :    →    ) (n : ) (x x : )
:= taylorWithinEval f n Set.univ x x

```

Criterio di Consistenza Assoluta (ACC): disuguaglianze algebriche rigorose per SBH e $F(R)$.

```

    [] theorem absolute_consistency_criterion_acc (SBH1 SBH2 SBHr : ) (h1
: 0 < SBH1) (h2 : 0 < SBH2) (h3 : SBHr = SBH1 + SBH2) : 0 < SBHr :=
by rw [h3] exact add_pos h1 h2

```

Teorema di compatibilità e regolarità geometrica analitica.

```

    [] theorem metric_and_manifold_consistency (M : Type*) [TopologicalSpace
M] (metric : MetricTensor M) : (g : M → (Fin 4 → Fin 4 → )), ( x, g x =
metric.g_components x) := by use metric.g_components intro x rfl
    end TdiProject.MathlibGeometryManifoldimport TdiProject.Basic import
TdiProject.SpinNetwork import TdiProject.Operators import TdiProject.Hamil-
tonian
    namespace TdiProject.Matter
    open TdiProject.Basic open TdiProject.SpinNetwork open TdiProject.Hamil-
tonian

```

Estensione dei gruppi di simmetria di gauge per includere il Modello Stan-
dard

```

    [] inductive GaugeGroup where | SU2 : GaugeGroup – Solo gravità (SU2)
| StandardModel : GaugeGroup – SU(3) x SU(2) x U(1) unificato

```

Rappresentazione dei fermioni o grani di carica localizzati sui nodi dello Spin
Network

```

    [] structure FermionicExcitation where node_id : Nat charge : Real
color_index : Nat – Rappresenta la carica di colore per SU(3)
    noncomputable section

```

Istanza Repr non computabile che evita di chiamare il repr sui reali

```

    [] instance : Repr FermionicExcitation := fun x _ => "FermionicEx-
citation(node_id := " ++ repr x.node_id ++ ", color_index := " ++ repr
x.color_index ++ ")"

```

Estensione dello stato geometrico per includere i campi di materia

```

    [] structure MatterCoupledGeometryState extends QuantumGeometryState
where gauge_symmetry : GaugeGroup fermions : List FermionicExcitation

```

Operatore di Interazione con la Materia H^{materia}

```

    [] def MatterHamiltonianOperator (state : MatterCoupledGeometryState)
: Real := match state.gauge_symmetry with | GaugeGroup.SU2 => 0.0 |
GaugeGroup.StandardModel => (state.fermions.length : Real) * 1.5e-35

```

Vincolo Hamiltoniano Totale: $\hat{H}_{\text{tot}} |\Psi\rangle = \hat{H}_{\text{LQG}} + \hat{H}_{\text{materia}} = 0$

```

[] def TotalWheelerDeWittConstraint (state : MatterCoupledGeometryState) : Prop := QuantumHamiltonianOperator state.toQuantumGeometryState + MatterHamiltonianOperator state = 0.0

```

Teorema: L'introduzione della materia preserva la consistenza del vincolo se bilanciata

```

[] theorem matter_coupling_consistency (state : MatterCoupledGeometryState) (h_base : WheelerDeWittConstraint state.toQuantumGeometryState) (h_matter_zero : MatterHamiltonianOperator state = 0.0) : TotalWheelerDeWittConstraint state := by unfold TotalWheelerDeWittConstraint rw [h_base, h_matter_zero] norm_num

```

Definizione di una fluttuazione o variazione dei campi di materia sui nodi

```

[] structure MatterFluctuation where original_state : MatterCoupledGeometryState perturbed_state : MatterCoupledGeometryState charge_conservation_law : original_state.fermions.length = perturbed_state.fermions.length

```

Teorema di Stabilità del Vincolo Hamiltoniano Totale sotto Fluttuazioni di Materia a Carica Conservata

```

[] theorem total_constraint_stability (fluc : MatterFluctuation) (h_orig : TotalWheelerDeWittConstraint fluc.original_state) (h_geom_invariant : QuantumHamiltonianOperator fluc.perturbed_state.toQuantumGeometryState = QuantumHamiltonianOperator fluc.original_state.toQuantumGeometryState) (h_matter_invariant : MatterHamiltonianOperator fluc.original_state = MatterHamiltonianOperator fluc.perturbed_state) : TotalWheelerDeWittConstraint fluc.perturbed_state := by unfold TotalWheelerDeWittConstraint at * rw [h_geom_invariant, ← h_matter_invariant] exact h_orig
end
end TdiProject.Matterimport Mathlib.Algebra.Ring.Basic import Mathlib.Topology.MetricSpace.Basic
namespace TdiProject.MatterSpinFoamAmplitudes

```

Struttura che rappresenta un bordo di un 4-plesso colorato con spin fermionici e di gauge.

```

[] structure SimplicialBoundaryColoring where fermionSpin : gaugeChargeLevel : is_valid_spin : fermionSpin 0.5 is_neutral_or_charged : gaugeChargeLevel 0

```

Regole di Feynman topologiche estese per i vertici Spin-Foam con accoppiamento alla materia.

```

[] noncomputable def TopologicalFeynmanVertexAmplitude (b : Simplicial-
BoundaryColoring) (couplingConstant : ) : := (b.fermionSpin ^ 2) * cou-
plingConstant / (1.0 + (b.gaugeChargeLevel : ))

```

Proprietà di conservazione delle cariche di gauge locali sui vertici del complesso simpliciale.

```

[] def LocalGaugeChargeConservation (incomingCharges outgoingCharges :
List ) : Prop := incomingCharges.sum = outgoingCharges.sum

```

Teorema di verifica della conservazione delle cariche di gauge locali nei processi di scattering Spin-Foam.

```

[] theorem gauge_conservation_proof (incoming outgoing : List ) (h_bal-
ance : incoming.sum = outgoing.sum) : LocalGaugeChargeConservation incom-
ing outgoing := h_balance
end TdiProject.MatterSpinFoamAmplitudes
import Mathlib.Topology.MetricSpace.Basic
import Mathlib.Analysis.ODE.PicardLindelof
import Mathlib.Analysis.Calculus.Deriv.Basic
import Mathlib.Analysis.Calculus.Taylor
set_option linter.unusedVariables false
namespace TdiProject.OpenAspectsLimitations

```

Struttura che formalizza le sfide infrastrutturali e tensoriali (es. uso di PhysLean, connessioni di Levi-Civita e identità di curvatura in formalismi generali).

```

[] structure TensorInfrastructureChallenge where requires_specialized_li-
braries : Bool uses_physlean : Bool rigorous_preliminary_definitions : Prop
is_satisfied_bool : Bool

```

Struttura che formalizza i calcoli quantistici e le anomalie conformi (es. derivazione delle anomalie di traccia per l'Entanglement Entropy e matching LQG-QFT).

```

[] structure QuantumAnomaliesChallenge where trace_anomalies_deriva-
tion : Prop non_perturbative_matching : Prop qft_formalization_pioneer_phase
: Bool

```

Teorema di coerenza formale per la rappresentazione degli aspetti aperti e limitazioni.

```

[] theorem open_aspects_and_limitations_consistency (tensor_chal : Ten-
sorInfrastructureChallenge) (quant_chal : QuantumAnomaliesChallenge) (h_ten-
sor : tensor_chal.rigorous_preliminary_definitions) (h_sat : tensor_chal.is_sat-
isfied_bool = true) (h_pioneer : quant_chal.qft_formalization_pioneer_phase

```

```

= true) : tensor_chal.is_satisfied_bool = true quant_chal.qft_formaliza-
tion_pioneer_phase = true := by constructor · exact h_sat · exact h_pi-
oneer
end TdiProject.OpenAspectsLimitationsimport TdiProject.Basic import TdiPro-
ject.SpinNetwork import Mathlib.Analysis.SpecialFunctions.Trigonometric.Ba-
sic
namespace TdiProject.Operators
open TdiProject.Basic open TdiProject.SpinNetwork

```

Autovalore dell'operatore di Area per uno spigolo con spin j : $A = 8 \sqrt{j(j+1)} P^2$

```

[] noncomputable def AreaEigenvalue (s : Spin) : Real := let j := (s.two_j
: Real) / 2.0 8.0 * Real.pi * ImmirziParameter * PlanckArea * Real.sqrt (j *
(j + 1.0))

```

Autovalore approssimato dell'operatore di Volume per un nodo v

```

[] noncomputable def VolumeEigenvalue (v : Node) (total_j : Real) : Real
:= (v.valence : Real) * PlanckVolume * Real.sqrt total_j

```

Simulazione dell'operatore di Olonomia $h_-(A)$ lungo un ciclo chiuso (Placchetta)

```

[] structure HolonomyOperator where loop_edge : Edge trace_value : Real
end TdiProject.Operatorsimport TdiProject.Basic import TdiProject.Spin-
Network import TdiProject.Operators
namespace TdiProject.SpectralGeometry
open TdiProject.Basic open TdiProject.SpinNetwork open TdiProject.Oper-
ators

```

Algebra di C^* -osservabili locali e funzioni scalari definite sui nodi della Rete di Spin

```

[] structure SpinAlgebra where dim_nodes : Nat node_values : Nat →
Real algebra_norm : Real h_norm_nonneg : algebra_norm 0

```

Operatore di Dirac Quantizzato $D(\Gamma, j, i)$ accoppiato alla geometria discreta di Spin Network

```

[] structure QuantizedDiracOperator where cutoff_scale_lambda : Real
spin_area_scale : Real h_lambda_pos : cutoff_scale_lambda > 0 h_area_pos
: spin_area_scale > 0

```

Tripletta Spettrale Quantizzata (A, H, D) secondo la Geometria Non-Commutativa di Connes

```

structure QuantumSpectralTriple where algebra : SpinAlgebra dirac_op
: QuantizedDiracOperator hilbert_dim : Nat h_dim_pos : hilbert_dim > 0

```

Valore del Commutatore $[D, a]$ tra l'operatore di Dirac e l'algebra delle osservabili

```

noncomputable def dirac_algebra_commutator (qst : QuantumSpectralTriple) (a : SpinAlgebra) : Real := a.algebra_norm * qst.dirac_op.spin_area_scale / qst.dirac_op.cutoff_scale_lambda

```

Teorema di Boundedness del Commutatore: Garantisce che $\|[D, a]\|$ sia limitata per ogni osservabile $a \in A$

```

theorem dirac_commutator_bounded (qst : QuantumSpectralTriple) (a : SpinAlgebra) : dirac_algebra_commutator qst a = 0 := by unfold dirac_algebra_commutator have h1 : a.algebra_norm = 0 := a.h_norm_nonneg have h2 : qst.dirac_op.spin_area_scale > 0 := qst.dirac_op.h_area_pos have h3 : qst.dirac_op.cutoff_scale_lambda > 0 := qst.dirac_op.h_lambda_pos have h_num : a.algebra_norm * qst.dirac_op.spin_area_scale = 0 := mul_nonneg h1 (le_of_lt h2) exact div_nonneg h_num (le_of_lt h3)

```

Valore dell'Azione Spettrale di Connes-Chamseddine $S_{\text{spec}} = \text{Tr}(f(D / \Lambda))$

```

noncomputable def spectral_action_value (qst : QuantumSpectralTriple) (f_zero_moment : Real) : Real := f_zero_moment * (qst.dirac_op.cutoff_scale_lambda ^ 4) * (qst.dirac_op.spin_area_scale ^ 2)

```

Teorema di Positività dell'Azione Spettrale Quantistica

```

theorem spectral_action_positivity (qst : QuantumSpectralTriple) {f_zero_moment : Real} (hf : f_zero_moment > 0) : spectral_action_value qst f_zero_moment > 0 := by unfold spectral_action_value have h_lam : qst.dirac_op.cutoff_scale_lambda > 0 := qst.dirac_op.h_lambda_pos have h_area : qst.dirac_op.spin_area_scale > 0 := qst.dirac_op.h_area_pos have h_lam4 : qst.dirac_op.cutoff_scale_lambda ^ 4 > 0 := by positivity have h_area2 : qst.dirac_op.spin_area_scale ^ 2 > 0 := by positivity have h_prod1 : f_zero_moment * (qst.dirac_op.cutoff_scale_lambda ^ 4) > 0 := mul_pos hf h_lam4 exact mul_pos h_prod1 h_area2

```

Teorema di Emergenza dell'Azione Einstein-Hilbert nel Limite Semiclassico: Dimostra che per scala UV $\Lambda \rightarrow 1$ l'Azione Spettrale domina e riproduce l'Azione classica $S_{\text{EH}} = \int \sqrt{g} dx$

```

    theorem spectral_action_semiclassical_limit (qst : QuantumSpectral-
    Triple) (f_zero_moment : Real) (hf : f_zero_moment > 0) (h_scale :
    qst.dirac_op.cutoff_scale_lambda ^ 4 > 1) : spectral_action_value qst f_zero_mo-
    ment f_zero_moment * (qst.dirac_op.spin_area_scale ^ 2) := by unfold spec-
    tral_action_value have h_area2 : qst.dirac_op.spin_area_scale ^ 2 > 0 := by
    positivity have h_step : f_zero_moment * (qst.dirac_op.cutoff_scale_lambda
    ^ 4) > f_zero_moment * 1 := by nlinarith nlinarith
    end TdiProject.SpectralGeometryimport Mathlib.Data.Real.Basic
    namespace TdiProject.SpinNetwork

```

Tipi di dato per identificare nodi e spigoli

```

    structure Node where id : Nat valence : Nat deriving DecidableEq, Repr
    structure Edge where id : Nat source : Node target : Node deriving
    DecidableEq, Repr

```

Spin j etichettato come semi-intero (es. $1 = 1/2$, $2 = 1$, $3 = 3/2$)

```

    structure Spin where two_j : Nat deriving DecidableEq, Repr

```

Rappresentazione del Grafo dello Spin Network

```

    structure SpinNetworkGraph where nodes : List Node edges : List
    Edge spin_assignment : Edge → Spin -- Vincolo di chiusura sui nodi (Gauss
    Constraint) gauss_verified : Bool

```

Predicato: Nodo ad alta valenza ($N > 4$)

```

    def is_higher_valence (n : Node) : Prop := n.valence > 4
    end TdiProject.SpinNetworkimport Mathlib.Data.Nat.Basic import Math-
    lib.Data.Int.Basic
    set_option linter.style.docString false set_option linter.style.whitespace false
    set_option linter.style.longLine false set_option linter.unusedVariables false
    namespace TdiProject.Topology
    inductive StrandType | over : StrandType | under : StrandType deriving
    DecidableEq, Repr
    structure Crossing where id : Nat strand : StrandType deriving Decid-
    ableEq, Repr
    structure DSTKnotState where crossing_count : Nat deriving Decid-
    ableEq, Repr
    inductive Move | flip (idx : Nat) : Move | r1 : Move | r2 : Move | r3 :
    Move deriving DecidableEq, Repr
    def is_valid_transition (s1 s2 : DSTKnotState) (m : Move) : Prop := match
    m with | Move.flip _ => s2.crossing_count = s1.crossing_count | Move.r1 =>

```

```

s2.crossing_count + 1 = s1.crossing_count | Move.r2 => s2.crossing_count +
2 = s1.crossing_count | Move.r3 => s2.crossing_count = s1.crossing_count
instance (s1 s2 : DSTKnotState) (m : Move) : Decidable (is_valid_transition s1 s2 m) := by unfold is_valid_transition cases m <;> dsimp <;> infer_instance

structure LaurentPoly where coeff : Int → Int deriving Inhabited
namespace LaurentPoly
def eq (p1 p2 : LaurentPoly) : Prop := k : Int, p1.coeff k = p2.coeff k
theorem eq_refl (p : LaurentPoly) : eq p p := fun _ => rfl
theorem eq_symm {p1 p2 : LaurentPoly} (h : eq p2 p1) : eq p1 p2 := fun k => (h k).symm
theorem eq_trans {p1 p2 p3 : LaurentPoly} (h1 : eq p1 p2) (h2 : eq p2 p3) : eq p1 p3 := fun k => (h1 k).trans (h2 k)
instance : Setoid LaurentPoly where r := eq iseqv := eq_refl, eq_symm, eq_trans
def zero : LaurentPoly := fun _ => 0
def add (p1 p2 : LaurentPoly) : LaurentPoly := fun k => p1.coeff k + p2.coeff k
def smul (c : Int) (p : LaurentPoly) : LaurentPoly := fun k => c * p.coeff k
end LaurentPoly
def kauffman_bracket (s : DSTKnotState) : LaurentPoly := fun k => if k = (s.crossing_count : Int) then 1 else 0
axiom kauffman_r1_invariant (s1 s2 : DSTKnotState) : is_valid_transition s1 s2 Move.r1 → LaurentPoly.eq (kauffman_bracket s1) (kauffman_bracket s2)
axiom kauffman_r2_invariant (s1 s2 : DSTKnotState) : is_valid_transition s1 s2 Move.r2 → LaurentPoly.eq (kauffman_bracket s1) (kauffman_bracket s2)
axiom kauffman_r3_invariant (s1 s2 : DSTKnotState) : is_valid_transition s1 s2 Move.r3 → LaurentPoly.eq (kauffman_bracket s1) (kauffman_bracket s2)
axiom kauffman_flip_invariant (s1 s2 : DSTKnotState) (idx : Nat) : is_valid_transition s1 s2 (Move.flip idx) → LaurentPoly.eq (kauffman_bracket s1) (kauffman_bracket s2)
inductive ReidemeisterPath : DSTKnotState → DSTKnotState → Type
where | refl (s : DSTKnotState) : ReidemeisterPath s s | step {s1 s2 s3 : DSTKnotState} (m : Move) (h : is_valid_transition s1 s2 m) (rest : ReidemeisterPath s2 s3) : ReidemeisterPath s1 s3
theorem kauffman_path_invariant {s1 s2 : DSTKnotState} (path : ReidemeisterPath s1 s2) : LaurentPoly.eq (kauffman_bracket s1) (kauffman_bracket s2) := by induction path with | refl s => exact LaurentPoly.eq_refl (kauffman_bracket s) | step m h _ ih => cases m with | flip idx => exact LaurentPoly.eq_trans (kauffman_flip_invariant _ _ idx h) ih | r1 => exact LaurentPoly.eq_trans (kauffman_r1_invariant _ _ h) ih | r2 => exact

```

```

LaurentPoly.eq_trans (kauffman_r2_invariant __ h) ih | r3 => exact Lau-
rentPoly.eq_trans (kauffman_r3_invariant __ h) ih
end TdiProject.Topology

```